

BLOC 2 - DOSSIER DE VALIDATION

Anonym

Developpement, tests, securite, accessibilite et deploiement

Application sociale web et mobile orientee messagerie temps reel, confidentialite, supervision et deploiement continu.

Candidat	Lukas Bouhlel
Version	v1.0.0-bloc2
Date	Juin 2026
Perimetre	Dossier principal limite a 30 pages maximum, hors annexes.

Dossier de validation - Bloc 2

Page de garde

Anonym est une application sociale web et mobile permettant aux utilisateurs de créer un compte, échanger en temps réel, gérer leur profil, rejoindre des conversations, utiliser une boutique virtuelle et accéder à des fonctionnalités de modération et d'administration.

Le projet repose sur une architecture composée :

- d'un backend Node.js / Express avec Socket.IO ;
- d'une base de données MySQL gérée avec Sequelize ;
- d'un frontend web React / Vite ;
- d'une application mobile Flutter ;
- d'une infrastructure Docker, Nginx et Certbot sur VPS ;
- d'une chaîne CI/CD GitHub Actions couvrant tests, sécurité, build, performance et déploiement.

SCREEN A AJOUTER

Capture de l'application web ou mobile sur l'écran principal.

SCREEN A AJOUTER

Capture du repository GitHub avec les branches `develop`, `preprod` et `main`.

Sommaire

1. Présentation du projet 2. Environnements de développement, test, préproduction et production 3. Intégration continue 4. Déploiement continu 5. Architecture logicielle et prototype 6. Tests unitaires, fonctionnels, sécurité et performance 7. Sécurité et OWASP 8. Accessibilité web et mobile 9. Versioning, recette et correction des anomalies 10. Documentation technique d'exploitation 11. Synthèse de validation des compétences du bloc 2

1. Présentation du projet

Anonym est une plateforme de communication orientée confidentialité. Elle permet à un utilisateur de s'inscrire, se connecter, gérer son profil, échanger dans des conversations, envoyer des messages texte et images, utiliser une carte avec Mapbox, gérer ses amis, accéder à une boutique et utiliser des fonctions de personnalisation.

Le projet cible deux supports principaux :

- le web, avec une interface React ;
- le mobile, avec une application Flutter Android.

Le backend centralise les règles métier, la sécurité, l'authentification, les données, les paiements, la messagerie temps réel et les métriques d'exploitation.

SCREEN A AJOUTER

Capture de l'écran de connexion web.

SCREEN A AJOUTER

Capture de l'écran de connexion mobile.

SCREEN A AJOUTER

Capture de la messagerie mobile avec un message texte et une image.

2. Environnements de développement, test, préproduction et production

2.1 Environnement de développement

Le développement est réalisé avec Visual Studio Code, Git, GitHub, Node.js, npm, Flutter, Android SDK, Docker, MySQL et les outils de test propres à chaque partie de l'application.

Les composants principaux sont :

Element	Choix technique	Role
Editeur	Visual Studio Code	Developpement et Remote SSH
Gestion de sources	Git + GitHub	Historique, branches, Pull Requests
Backend	Node.js / Express	API REST, securite, Socket.IO
Base de donnees	MySQL + Sequelize	Persistence, migrations, modeles
Frontend web	React / Vite	Interface web utilisateur et admin
Mobile	Flutter	Application Android
Deploiement	Docker Compose	Execution des services en preprod/prod
Reverse proxy	Nginx	HTTPS, routage, WebSocket
Certificats	Certbot	TLS en production
Monitoring externe	OVH + Site24x7	Disponibilite et supervision serveur

SCREEN A AJOUTER	Capture de VS Code avec l'arborescence du projet.
SCREEN A AJOUTER	Capture de Docker ou du serveur montrant les conteneurs actifs.

2.2 Environnements cibles

Le cycle de livraison est separe en plusieurs environnements :

- developpement local : execution locale du backend, frontend web et Flutter ;
- integration continue : execution automatique des tests dans GitHub Actions ;
- preproduction : deploiement automatique apres validation CI de la branche `preprod` ;
- production : deploiement automatique apres validation CI de la branche `main` .

Cette separation permet de tester les evolutions avant leur mise en production et de limiter les regressions.

SCHEMA A AJOUTER	Schema simple : Developpement local -> GitHub -> CI -> Preproduction -> Production.
------------------	---

3. Integration continue

3.1 Objectif

L'integration continue permet de verifier automatiquement la qualite du code a chaque modification. Elle reduit le risque de regression en executant les controles avant les deploiements.

La CI est definie dans `.github/workflows/ci.yml` .

Elle se declenche :

- a chaque push ;
- sur les Pull Requests vers `preprod` .

3.2 Pipeline CI

La CI est organisee sous forme de graphe GitHub Actions avec des dependances entre jobs.

Ordre logique :

1. Prepare CI 2. Security checks 3. Flutter mobile validation 4. Web and backend validation 5. Backend performance validation 6. CI passed

Les controles realises sont :

- audit de securite Snyk frontend, backend et Flutter ;
- `npm audit` frontend et backend ;
- lint frontend, SCSS et backend ;
- generation de documentation JSDoc ;
- tests unitaires frontend ;
- tests unitaires backend ;

- analyse Flutter ;
- tests Flutter ;
- build Android APK release signe ;
- test de performance backend avec Artillery.

SCREEN A AJOUTER

Capture du graphe GitHub Actions de la CI avec les jobs relies.

SCREEN A AJOUTER

Capture d'une execution CI verte.

SCREEN A AJOUTER

Capture du job Flutter montrant le build APK release.

3.3 Justification

Cette CI répond aux exigences du bloc 2 car elle teste regulierement les blocs de code, verifie la securite des dependances, valide le build web, backend et mobile, et fournit une preuve automatique avant le deploiement.

4. Deploiement continu

4.1 Strategie de branches

Le projet utilise une strategie progressive :

Branche	Role
devlop	Developpement courant
preprod	Validation avant production
main	Production

Le flux de livraison est :

devlop -> preprod -> main

La branche main represente la version stable deployable en production.

SCREEN A AJOUTER

Capture GitHub du reseau de branches ou des Pull Requests devlop -> preprod et preprod -> main .

4.2 CD preproduction

Le workflow `.github/workflows/ci-cd.yml` deploye la preproduction apres succes de la CI sur preprod .

La preproduction permet de verifier l'application dans un environnement proche de la production avant de merger vers main .

SCREEN A AJOUTER

Capture d'une execution verte du workflow de preproduction.

4.3 CD production

Le workflow `.github/workflows/cd.yml` deploye la production uniquement apres succes de la CI sur main .

Le pipeline production suit maintenant une chaine graphique :

1. Prepare production deployment 2. Build signed mobile APK 3. Deploy web and backend to VPS 4. Publish APK to VPS

La CD production :

- attend que la CI soit terminee et reussie ;
- build un APK Android release signe ;
- deploye le backend et le web sur le VPS ;
- execute Docker Compose en production ;
- publie l'APK sur le VPS dans `/home/lukas/mobile-artifacts/anonym` ;
- conserve les 10 derniers APK ;
- evite d'ajouter l'APK dans le repository GitHub.

SCREEN A AJOUTER	Capture du graphe GitHub Actions de la CD production.
SCREEN A AJOUTER	Capture du dossier VPS contenant les APK generees.
SCREEN A AJOUTER	Capture des conteneurs Docker apres deploiement.

4.4 Gestion des erreurs de deploiement

Le deploiement production verifie l'etat du repository sur le VPS. Si le serveur contient des changements locaux non sauvegardes, la CD s'arrete pour eviter d'ecraser des fichiers.

Si la branche `main` locale du serveur a diverge de `origin/main`, le workflow cree une branche de sauvegarde avant de realigner le serveur sur la version distante.

Cela permet de garder une production stable tout en conservant une trace des commits locaux presents sur le serveur.

5. Architecture logicielle et prototype

5.1 Architecture generale

L'architecture d'Anonym est separee en plusieurs couches :

- presentation : React et Flutter ;
- orchestration client : contexts React, providers Flutter ;
- services clients : appels API, Socket.IO, repositories ;
- API backend : routes, middlewares, controleurs ;
- domaine et persistance : modeles Sequelize, migrations, MySQL ;
- infrastructure : Docker, Nginx, Certbot, VPS.

UML / SCHEMA A AJOUTER	Schema d'architecture globale C4 ou equivalent : Utilisateur -> React/Flutter -> Nginx -> API Express/Socket.IO -> MySQL/uploads -> services externes.
UML A AJOUTER	Diagramme de sequence d'authentification securisee.
UML A AJOUTER	Diagramme de sequence d'envoi d'un message temps reel.
UML A AJOUTER	Diagramme de classes ou MCD simplifie : User, Channel, UserChannel, PrivateMessage, Friend, Shop, Inventory, Invoice.

5.2 Prototype web et mobile

Le prototype est fonctionnel sur web et mobile. Il couvre les principales fonctionnalites :

- inscription et connexion ;
- gestion du profil ;
- messagerie temps reel ;
- envoi de messages texte et images ;
- navigation principale ;
- gestion des amis ;
- carte Mapbox en mode nuit ;
- boutique ;
- administration web pour les utilisateurs et la boutique ;
- monitoring technique du backend.

SCREEN A AJOUTER	Capture du dashboard ou de l'accueil web.
SCREEN A AJOUTER	Capture de la navigation mobile.
SCREEN A AJOUTER	Capture de la map mobile en mode nuit.
SCREEN A AJOUTER	Capture de la boutique.
SCREEN A AJOUTER	Capture de l'administration web.

6. Tests unitaires, fonctionnels, securite et performance

6.1 Tests unitaires

Le projet contient des tests automatises sur plusieurs parties :

- tests backend avec Jest et Supertest ;
- tests frontend web ;
- tests Flutter ;
- tests d'accessibilite mobile via widgets Flutter ;
- tests d'observabilite backend pour `/health` , `/status` , `/api/health` et `/metrics` .

Les tests sont executes automatiquement dans la CI.

SCREEN A AJOUTER	Capture du job CI backend montrant les tests en succes.
SCREEN A AJOUTER	Capture du job CI Flutter montrant <code>flutter test</code> en succes.

6.2 Tests de performance backend

Le backend expose des routes de disponibilite et de metriques :

- `/health`
- `/status`
- `/api/health`
- `/metrics`

Un test de performance Artillery est configure dans :

`anonym-back-end/app/tests/performance/health.yml`

Il verifie :

- le taux d'erreur maximal ;
- le temps de reponse p95 ;
- le temps de reponse maximal ;
- la disponibilite des endpoints de healthcheck ;
- la disponibilite des metriques Prometheus.

SCREEN A AJOUTER	Capture du job <code>Backend performance validation</code> dans GitHub Actions.
---------------------	---

**SCREEN A
AJOUTER**

Capture de la sortie Artillery avec les temps de reponse.

6.3 Observabilite

Le backend utilise :

- `pino-http` pour les logs structures ;
- `prom-client` pour les metriques Prometheus ;
- des endpoints de sante pour le monitoring ;
- Site24x7 et OVH pour la supervision externe.

Les metriques techniques permettent d'observer la disponibilite, les temps de reponse et le comportement du serveur.

**SCREEN A
AJOUTER**

Capture de `/health` dans le navigateur ou via curl.

**SCREEN A
AJOUTER**

Capture de `/metrics` montrant les metriques Prometheus.

**SCREEN A
AJOUTER**

Capture Site24x7 ou OVH montrant la surveillance du serveur.

7. Securite et OWASP

7.1 Mesures de securite mises en place

Le projet integre plusieurs mesures de securite :

- authentification par JWT ;
- refresh token ;
- cookies `httpOnly` ;
- protection CSRF ;
- CORS configure par environnement ;
- Helmet et Content Security Policy ;
- rate limiting et ralentissement des requetes sensibles ;
- controle des roles administrateurs ;
- autorisation par ressource ;
- verification des droits sur les messages, channels, amis, boutique et administration ;
- durcissement des uploads ;
- logs sans donnees sensibles ;
- erreurs de production moins bavardes ;
- secrets stockes dans GitHub Secrets et variables d'environnement ;
- audit des dependances avec Snyk et `npm audit` ;
- rotation des tokens prevue dans la politique d'exploitation ;
- HTTPS via Nginx et Certbot.

**SCREEN A
AJOUTER**

Capture des GitHub Secrets, sans afficher les valeurs.

**SCREEN A
AJOUTER**

Capture d'un audit Snyk ou npm audit en succes.

**SCREEN A
AJOUTER**

Capture du certificat HTTPS sur le site en production.

7.2 Couverture OWASP Top 10

Risque OWASP	Reponse projet
A01 Broken Access Control	Middleware auth/admin, verification des permissions par ressource
A02 Cryptographic Failures	HTTPS, cookies httpOnly , secrets hors code source
A03 Injection	ORM Sequelize, validation des entrees, sanitization
A04 Insecure Design	Architecture en couches, separation admin/utilisateur, controles metier
A05 Security Misconfiguration	Helmet, CORS, variables par environnement, Docker/Nginx
A06 Vulnerable Components	Snyk, npm audit, CI de securite
A07 Identification and Authentication Failures	JWT, refresh token, mots de passe hashes
A08 Software and Data Integrity Failures	Git, CI/CD controlee, APK signe, dependances auditees
A09 Security Logging and Monitoring Failures	Logs structures, healthchecks, metriques Prometheus, Site24x7
A10 Server-Side Request Forgery	Limitation des integrations externes, controle des URLs et services utilises

TABLEAU A
AJOUTER DANS
CANVA

Reprendre ce tableau OWASP en 2 colonnes avec une couleur par niveau de couverture.

8. Accessibilite web et mobile

8.1 Referentiel choisi

Le referentiel retenu est le RGAA, car il est adapte au contexte francais et permet d'evaluer l'accessibilite des interfaces numeriques. Pour le mobile Flutter, les principes sont appliques via les recommandations d'accessibilite Flutter : labels semantiques, navigation lisible, zones tactiles explicites, compatibilite lecteurs d'ecran et informations non uniquement visuelles.

8.2 Actions mises en place

Sur mobile, des labels accessibles et des Semantics ont ete ajoutes sur les ecrans importants :

- connexion ;
- inscription ;
- navigation principale ;
- messages ;
- profil.

Les tests widgets prouvent la presence des labels accessibles :

```
anonym_front_flutter/test/screens/accessibility_semantics_test.dart
```

Les elements verifies incluent :

- champs E-mail ou pseudo ;
- champ Mot de passe ;
- action Connexion ;
- action d'affichage du mot de passe ;
- consentement aux conditions generales ;
- navigation Accueil , Messages , Recherche , Profil , Creer ;
- actions de messagerie Ajouter une image , Message , Envoyer le message .

SCREEN A AJOUTER	Capture du test Flutter accessibilite en succes.
SCREEN A AJOUTER	Capture de l'ecran login mobile.
SCREEN A AJOUTER	Capture de l'ecran inscription mobile.

SCREEN A AJOUTER	Capture de la navigation mobile.
SCREEN A AJOUTER	Capture de la messagerie mobile.
SCREEN A AJOUTER	Capture du profil mobile.

8.3 Benefices pour les utilisateurs

Ces actions facilitent l'utilisation de l'application pour :

- les personnes utilisant un lecteur d'ecran ;
- les personnes ayant des difficultes visuelles ;
- les personnes ayant besoin de libelles clairs pour comprendre les actions ;
- les utilisateurs ayant des difficultes motrices, grace a des actions identifiabiles ;
- les utilisateurs ayant besoin d'une navigation simple et coherente.

9. Versioning, recette et correction des anomalies

9.1 Gestion de versions

Le projet utilise Git et GitHub pour tracer les evolutions. Les changements sont organises par branches, Pull Requests et workflows CI/CD.

Un tag de version permet d'identifier la version presentee pour le bloc 2 :

v1.0.0-bloc2

SCREEN A AJOUTER	Capture de l'historique Git ou du tag v1.0.0-bloc2 .
SCREEN A AJOUTER	Capture des Pull Requests mergees vers preprod et main .

9.2 Cahier de recette

Scenario	Resultat attendu	Preuve
Inscription utilisateur	Le compte est cree et l'utilisateur peut confirmer son inscription	Screenshot inscription
Connexion utilisateur	L'utilisateur accede a son espace	Screenshot login reussi
Modification profil	Les informations de profil sont mises a jour	Screenshot profil
Ajout ami	La demande est envoyee et visible	Screenshot amis
Messagerie texte	Le message apparait en temps reel	Screenshot chat
Messagerie image	L'image apparait en temps reel chez les autres utilisateurs	Screenshot chat image
Modification/suppression message	Le changement est synchronise en temps reel	Screenshot avant/apres
Map mobile	La carte reste en mode nuit	Screenshot map
Boutique	Les articles sont visibles et achetables	Screenshot boutique
Administration	Les fonctions admin restent reservees aux admins	Screenshot acces admin
Healthcheck backend	/health renvoie un statut OK	Screenshot /health
Metriques backend	/metrics renvoie des metriques Prometheus	Screenshot /metrics
CI	Tous les jobs passent	Screenshot CI verte
CD	Le deploiement prod passe apres la CI	Screenshot CD verte

9.3 Plan de correction des bogues

Anomalie detectee	Cause probable	Correction appliquee	Statut
Acces interdit sur certaines fonctions mobile non admin	Controle de role trop large ou mauvaise route appelee	Verification des permissions backend/mobile et separation admin/utilisateur	Corrige
Image de chat non visible en temps reel pour les autres utilisateurs	Evenement Socket.IO incomplet pour les messages avec image	Synchronisation temps reel des messages texte + image	Corrige
Map parfois en mode clair	Style Mapbox non verrouille	Mode nuit force sur la carte	Corrige
Photo de profil non synchronisee	Rafraichissement profil non diffuse	Mise a jour temps reel de l'avatar	Corrige
Modification/suppression message non synchronisee	Evenements temps reel manquants	Ajout de la synchronisation des editions/suppressions	Corrige
CD production lancee en meme temps que la CI	Trigger trop direct sur merge PR	Passage a workflow_run apres CI reussie	Corrige
Repository serveur diverge	Commits locaux sur le VPS	Backup branch puis realignement controle sur origin/main	Corrige

SCREEN A AJOUTER

Capture d'une issue, d'un commit ou d'une PR montrant une correction.

10. Documentation technique d'exploitation

10.1 Manuel de deployment

Le deployment repose sur Docker Compose, Nginx, Certbot et GitHub Actions.

Commandes utiles :

```
docker-compose -f docker-compose.prod.yml --env-file .env.production up -d --build --remove-orphans
```

La CD production execute automatiquement cette commande apres validation CI.

SCREEN A AJOUTER

Capture du fichier .github/workflows/cd.yml .

10.2 Manuel d'utilisation

L'utilisateur peut :

- creer un compte ;
- se connecter ;
- modifier son profil ;
- ajouter des amis ;
- discuter en temps reel ;
- envoyer des images ;
- utiliser la carte ;
- consulter la boutique ;
- acheter des elements ;
- consulter ses factures ou son inventaire.

L'administrateur peut :

- gerer les utilisateurs ;
- gerer la boutique ;
- acceder aux fonctions d'administration ;
- superviser les contenus selon les droits prevus.

SCREEN A AJOUTER

Parcours utilisateur complet en 4 captures : login -> navigation -> message -> profil.

10.3 Manuel de mise a jour

Procedure de mise a jour :

1. Developper sur `develop` . 2. Ouvrir une Pull Request vers `preprod` . 3. Attendre le succes de la CI. 4. Valider le deploiement preproduction. 5. Ouvrir une Pull Request `preprod` vers `main` . 6. Attendre le succes de la CI sur `main` . 7. Laisser la CD production deployer le VPS. 8. Verifier la production, les healthchecks et les logs.

SCHEMA A AJOUTER

Schema de workflow Git : `develop -> preprod -> main -> VPS` .

10.4 Exploitation et stockage VPS

Une documentation d'exploitation stockage existe dans :

`docs/exploitation-stockage-vps.md`

Elle decrit :

- le diagnostic disque ;
- le nettoyage controle des backups ;
- la retention recommandee ;
- l'objectif de garder le disque sous 90 % ;
- la prevention des erreurs Remote SSH liees au manque d'espace.

SCREEN A AJOUTER

Capture de `df -h` sur le VPS.

SCREEN A AJOUTER

Capture du dossier de backups apres nettoyage.

11. Synthese de validation des competences du bloc 2

Competence	Reponse du projet	Preuves a fournir
C2.1.1 Environnements de deploiement et test	Environnements local, CI, preprod, prod, monitoring, performance	Screens CI/CD, Docker, Site24x7, <code>/health</code>
C2.1.2 Integration continue	GitHub Actions avec jobs securite, tests, build, Flutter, performance	Graphe CI, execution verte
C2.2.1 Prototype logiciel	Web React, mobile Flutter, backend Express, Socket.IO, MySQL	Screens web/mobile, UML architecture
C2.2.2 Tests unitaires	Jest, Supertest, tests frontend, tests Flutter, tests accessibilite	Screens tests CI
C2.2.3 Securite et accessibilite	OWASP, CSRF, JWT, Helmet, Snyk, Semantics Flutter, RGAA	Tableau OWASP, tests accessibilite
C2.2.4 Deploiement progressif	Branches <code>develop</code> , <code>preprod</code> , <code>main</code> , CD preprod/prod, versioning	Screens branches, PR, CD
C2.3.1 Cahier de recette	Scenarios fonctionnels, techniques, securite et performance	Tableau recette + captures
C2.3.2 Correction des bogues	Anomalies identifiees, qualifiees et corrigees	Tableau anomalies + commits/PR
C2.4.1 Documentation exploitation	README, docs architecture, exploitation VPS, CI/CD, deploiement	Captures documentation

Conclusion

Le projet Anonym repond aux attendus du bloc 2 grace a une application fonctionnelle web et mobile, une architecture maintenable, une integration continue complete, un deploiement progressif vers preproduction puis production, des tests automatises, des controles de securite, des mesures d'accessibilite mobile et une documentation d'exploitation.

Les preuves a fournir dans le PDF sont principalement des captures GitHub Actions, des captures applicatives, des diagrammes UML/C4, des endpoints de monitoring et des traces de versioning Git. Ces elements permettront de demontrer que le projet est stable, securise, testable, deployable et exploitable.